Back to the case

Edition compass · Case 02 · Albrecht · Joint version 1.1 · 27 October 2026 · Material

# We can describe this in more detail.

*A better interpretation changes more than a day in the text.*

Nora comes across a letter post that insufficiently captures the previous variant model. She wants to change the markup. Not for the pleasure of conversion: The new distinction makes her scientific argumentation more precise.

Samir opens the preview. The new proposal also concerns register references, web presentation and sentence. Previous volumes have been published with a different model version. The question is not only whether Nora is right, but how her improvement can be included in the overall edition.

The Commission has developed a limited experiment. The team compares change only for future volumes, retrospective migration and parallel model versions. Each variant preserves something and creates new work.

The decided model change is documented in ODD, examples and manual with scope. Old band stands remain comprehensible with their then rules.

> "The schema must not decide what I can say scientifically." — "All right, but we also need to know how to spend this statement in ten years."

**How does Git help?**

A test branch keeps the new modeling trialable separately. Versioned data, schemas and migration scripts make differences understandable. A Git tag designates a specific version; a tape release also requires the appropriate index and software levels.

**What is there to consider?**

Which old stocks are affected? Who checks the semantic correctness of a migration? Is a new schema sufficient or does the edition report have to change?

**Action**

Choose a representative test passage, document the impact and care costs, examine the technical and technical aspects and keep the decision with its scope.

**Responsible role**

Bandredaktion, scientific-technical editorial and authorized edition commission

**Institutional question:** Who finances the improvement, including work on earlier volumes?

# The report's here.

*A fixed version allows a judgment. It does not end the discussion.*

The test bench has been sent out, the publisher has provided a production window. Then the external expert opinion is received: A central text decision is not yet sufficiently substantiated.

Nora takes a hint, contradicts a second and revises a third section. Eva organizes who decides on the answer and what changes need to be re-evaluated. The reviewer should not have to learn the Git system or see the entire internal work history.

The editors provide a readable HTML or PDF extract with precise stand identification. Feedback remains in the agreed confidential area. The result is documented well-founded; a technical examination alone does not decide any scientific issue.

The manual determines which test bench and which standard version belong to the report. Confidential feedback is in the separate review area; professional answers are documented on a reasoned basis.

> "We can't just put in every criticism." — "No. But we have to be able to show how we tested it."

**How does Git help?**

A clearly marked test bench and the comparison to the revision combine feedback and editing. Protected test packages can be generated from versioned sources.

**What is there to consider?**

Who is allowed to read the report? What anonymity is agreed? What change does a re-examination require? Sectors are not a reliable separation of reading permissions.

**Action**

Check stand, back channel, competent authority and deal with revisions; coordinate schedules with the publisher.

**Responsible role**

Expert person, band editor, general editor and publisher's editor

**Institutional question:** Who carries additional review time and decides on remaining dissent?

# The data would be ideal for our research

*Research interest meets an already agreed publication strategy.*

A research team wants to use commented letter data for a new network analysis. The collaboration promises insights and makes the edition visible early, but the volume in question is not yet open.

Clara recalls that the publisher invests in editing, set and production. Delayed digital delivery was part of the agreement. Eva wants to enable cooperation without passing any promises.

The team separates metadata, text, comment, images and publishing PDF. What use is already allowed, which one needs an additional agreement? A deadline alone does not answer this question. The case does not require a specific contract solution.

The rights/publication office uses the release procedure from the manual. Research on released letter data and a later access to the internal history of creation are different requests.

> "The data is something different from the book." — "Are they? We need to know what we actually agreed with each other. "

**How does Git help?**

After an agreed approval, a curated data package can be accurately identified and published in its own public repo. Private history and confidential feedback will not be exported.

**What is there to consider?**

Which items does the contract include? Which third-party materials have their own terms and conditions? What is only accessible, what can also be reusable? Who may make a binding decision?

**Action**

Check concrete agreements, separate types of data, negotiate additional access if necessary and document each approval in a comprehensible manner.

**Responsible role**

Project management, competent legal advice, authorised bodies and publishing house

**Institutional question:** What support does research and a sustainable publication allow?

# Volume 7, please repeat

*In 2055, the old host disappeared.*

A new working group wants to reprint an older volume. The database at that time is no longer in operation, the original Git host no longer exists. A book book book and a data package are preserved.

The package contains a manifest: it lists text status, schema, ZX Q1 snapshot and output program. The group checks whether these components actually exist. A list alone does not replace the files.

Where the previous edition can be reconstructed, the team documents the way. Where something is missing, it marks the gap. A new edition receives its own responsible version. The previous publication will not be reproduced afterwards as unchanged.

In the archive package, the group also finds the then manual and profile description. It is only with this that it becomes clear which tests and handovers belonged to this bandstand.

> "We have the texts. " — "Good. But do we also have the index to which these texts refer?"

**How does Git help?**

A transferable version history preserves file versions and its relationships. It can facilitate restart, but does not replace DB snapshots, context, rights decisions, and available transformations.

**What is there to consider?**

What is actually received? Can output and references be tested? Which platform data have been taken over separately? Who can release the new issue?

**Action**

Check archive packages for completeness, regularly test restoration, document missing components and institutionally secure transfers.

**Responsible role**

Infrastructure providers, archive services and new scientific editors

**Institutional question:** What recurring tests would have made the gaps visible earlier?

BETWEEN SITUATION 04 AND 05

## The edition learns with every band

For decades, the workflows of an edition have not stopped. Recurring steps can be better organized with experience; at the same time, growing index and more cross-references make new solutions necessary. The editorial team tests routines, takes on tried and tested and improves. Each introduced change receives a suitable manual version; rework on older collections is documented as its own order.

Technical progress is also being made: OCR can provide templates, for example LLM can provide proposals for annotations. Such results are subject to technical examination. New rules should also be applied to older data without overwriting previously done work.

Git helps to keep data, rules and processing scripts together in suitable stands. The editors can first test a change separately, compare results and understand which old data have already been traced. Whether the improvement is technically correct and the handovers work, they must continue to check. That is exactly what the next situation is all about.

# The typesetting proof was already tested

*A popular new surface facilitates commenting. Samir discovers how her return journey changes already tested work again.*

**"We've already corrected that!"**

Sometimes a distance is not right, sometimes a line, sometimes a reference to the register. At first every finding seems small for itself. Together they stall the book production. The publishing house paid for an advertising campaign; just in Albrecht's anniversary year a band threatens to fail again. Clara blames the typesetting service provider.

This contradicts: the delivered data looks sloppy. Countless little things would even be tacitly improved in typesetting. "Can no one explain to the editors what a non-breaking space is?

The editors are outraged. They have carefully marked distances and stroke types and checked the relevant places immediately afterwards. Nora remembers exactly a correction that is wrong again in the flag. From the editor's point of view, the error must have come into the typesetting.

Samir listens to the dispute. He promises to look at the history of the affected places: Maybe it is possible to find out when and how the mistakes have come back.

**Samir's first track: the late processing**

Samir explains how he started: "I have put together all the late corrected passages. Then I looked: What were the last changes before the correction? And what are the arrangements between the first occurrence of the error and its remedy? What do these paths have in common?

In his selection the associated timestamps are mostly late at night. He shows some entries. "And anyway: The closer the deadline moves, the further the entries reach into the night. At first I thought we had a concentration problem and maybe should change something about our biorhythms.

> An excerpt from Samir's view
>
> 1. 04.06.**00:27**Comment balanced with index

2. 06.06.**00:37**Replayed revised comments
3. 08.06.**02:30**Register reconciliation completed
4. 09.06.**03:35**Corrections for the next flag accepted

Exemplary commit timestamp from the case history.

"But it wasn't," he continues. "The late timestamps didn't explain the cause. I had to take a closer look at the places themselves and their prehistory." git blame he finds an entry into the assigned change; with git show and the comparison of two versions he reads further back. Characters, annotation and test deduction he looks together.

The editorial staff remembers correctly: the special distances and the correct line were already stored correctly. Samir changes between this near reading and the comparison of many cases. In the overview he finds a common way of editing. In the individual versions he sees what is lost on this way.

---

▼ git blame: What is the change behind this line?

git blame assigns to each line of a selected file version the change it last changed. Samir gets a commit identifier and can open the corresponding change.

This is an introduction to the prehistory: A later reformatting can also appear as a last change. The command does not explain the technical cause nor any personal responsibility. Therefore Samir reads on with git show and version comparisons.

```
git blame -L 42,48 -- kommentare.xml
```

Shows the change assignment for lines 42 to 48 of the file kommentare.xml.

Git documentation: blame

---

▼ git show: What was changed in this step?

git show opens a selected commit: its description and, in the case of an ordinary commit, changes to the predecessor. Samir sees which files and text passages have been changed together.

With the commit identifier from git blame he follows the return import. For the direct comparison of two selected versions he also uses git diff. The technical reading of the XML positions remains part of the investigation.

```
git show a1b2c3d -- kommentare.xml
```

a1b2c3d is an example of a commit identifier. The change to kommentare.xml is shown.

Git-Documentation: show

Git: blemish · Git: show · Git: Compare Differences

**The new comment interface: a great relief**

For some time now there has been a new interface for comments that need to be compared with the index. On the left is the readable comment, on the right there are matching person, location and work records. The editors can compare hits and select the correct link without constantly switching between XML files and register lists.

Everyone confirms the profit. The work is easier to do, confusion can be resolved immediately. "We definitely want to keep this surface," says Nora. Nobody wants to go back to the laborious earlier way of working.

In this successful new way, Samir finds the cause. When he is involved in the project, comments are transferred to the working view and then played back. The back import simplifies certain special distances and strokes. Furthermore, the connection does not carry existing date distinctions without loss: it reconstructs them afterwards with the number recognition from the returned text.

There are two things that remain invisible in the comfortable surface: A carefully set range dash is changed – and the markup recalculated from it now says something different. The surface is a real help. Your way back needs a repair.

WORKFLOW'S KEY POWER

**Comment more conveniently. Information is lost when playing back.**

The progress is undisputed: Comment and matching index data stand next to each other. The error lies in the integration of this new work path.

COMMENTARY · LESBARE WORKING ASPECTS

Albrecht's correspondence developed in 1848–49 in close association with his brother.

Match recipients and register receipts directly to the text.

COMPLAINTING REGISTER DATA

Person: Albrecht's brother

Period: 1848–1849

Matching directly to the comment

> "We definitely want to keep this surface.

**1. Tested TEI data**

Markers and period are already stored correctly.

**2. Readable comment view**

Only for comments with register reconciliation. The technical processing becomes easier.

**3. Wrong way back**

Characters are simplified; date distinctions from the text are recalculated.

Before exporting properly – changed after reimporting. An earlier check cannot prevent this later loss.

Schematic view of the work path used in the case.

**Why it just hits some comments**

The remaining comments will continue to be processed in the previous XML tool. Only the comments with register matching take the path through the new interface. Therefore, the errors do not appear everywhere.

Even under these comments, many remain inconspicuous: if you do not contain any of the affected distances or strokes, you can pass the problematic return route without a visible accident. Only the meeting of processing path and relevant characters triggers the described chain.

Samir can now explain the many individual findings. "Take a closer look at your characters" the editors had done long ago. The markers had been checked – before the later step, which simplified them again.

### From one stroke a register cascade is created

A small example shows the sequence. The commentary says "1848–1849, excellent as a contiguous period. An automated step detects such time data and inserts an annotation layer for the index. Project-internal number recognition also uses the ZX Q1. After the return journey there is "1848-1849: When the layer is rebuilt, the rule now generates two individual annual nominations. The previous period markup will not be received.

The rule creates formally permitted, but differently assigned data. Register entries and the web finder are derived from it. At the same time lost protected distances can change the change. In the flag someone finds the wrong line and repairs it. A index already released for the sentence no longer fits the corrected text and output level.

Not only the flag is affected. Also the separate web finder, register extracts and already prepared register volumes must be checked for the affected changes and possibly re-created. Stable IDs prevent some reference shift, but no wrong temporal annotation or an outdated page reference.

The dependence lies here in the agreed project rules and output routes. Neither an ordinary line nor TEI leads on its own to this interpretation. This is exactly what Samir needs to check.

## Thus, from a period of time something different

#### Before the process: a period

```
<date from="1848" to="1849">1848–1849</date>
```

from and to describe the beginning and end of the associated period.

#### After the wrong way back and the re-marking

```
<date when="1848">1848</date>-<date when="1849">1849</date>
```

When here each describes a single year. The connection as a period is missing. Both fragments can be allowed as a structure; the statement is not the same.

A mere replacement of the visible line within the original date markup would not delete from and to. In the case of the faulty connection, the original annotation does not result in lossless and rebuilds it from the simplified text. Therefore, Samir must also examine export, import and number recognition together.

XML sections in the TEI context. TEI explains date and date attributes

**A selective detour, several dependent expenditures**

The new interface facilitates register matching. Only part of the comments go through it. The way back changes certain markers and reconstructs date annotation from the text; exactly this combination triggers the error.

This comment is compared with the index data. It contains affected spacings or bar signs.

Both conditions come together: The return journey changes relevant characters. Number recognition, index and dependent outputs can therefore run apart.

**Marker and period correct**

"1848–1849" is recognized as a period of time. Typographic markers are tested; the index matches this data level.

**Targeted control:** Repair the way back; check the affected markers, annotations and expenditures after the comparison. Keep corrections in the relevant inventory and create dependent stands together again.

**Luke corrects a factual error – and triggers the relapse**

It is particularly clear from a comment by Luke. The wrong line had already been discovered and repaired. The correction was returned to the relevant data set; the index was re-examined and released for the set.

Then Luke reads the next flag and discovers a real mistake of his own: He confused the recipient of a letter. He opens the comment in the familiar new surface, corrects the recipient and selects the appropriate person entry from the index. A correct and necessary correction.

After playing back, the range dash, which has been repaired before, has once again become a simple "-" . Lukas has not touched the relevant time. Nevertheless, the connection recalculates its annotation. Time period and register assignment are again not correct; the cascade reaches sentence, web finder and register output again.

"But I only corrected the receiver!" says Lukas. This is exactly what makes the case for Samir so instructive. Not the new technical correction was wrong. It triggered the same wrong way back again. The seemingly careless causer unintentionally reproduced the mechanism.

**The correct correction takes the wrong path again**

1. **Already repaired**

   Line and period are correct again; index and sentence level are checked.

2. **Luke discovers a material error**

   He corrects the recipient and rebalances the comment with the index.

3. **Untouched Time Changed**

   The return import simplifies the line and creates the date mark again.

4. **The cascade is back**

   index, set, web finders and register volumes must be checked together again.

**The revealing difference:** Luke corrects the contents correctly. His correction triggers the wrong transmission route again.

**What kind of workflow problem is that?**

Samir distinguishes the lossy route of transmission, the unfavorable sequence of examination and subsequent change, the passing on into dependent expenditures and the repeated repair of only visible symptoms. Such weaknesses can interact; not every error has the same cause.

The immediate countermeasure lies after the reimport: The editorial team checks signs, annotation and output there. This additional control is immediately recorded in the manual. Parallel to this, the incorrect connection of the comment tool is revised – so that the tested content will survive the whole way unchanged in the future.

Technical corrections go back to the relevant data stock. From this, affected annotations and issues are re-derived in a controlled manner. A release state documents text, index, rules and output path together. Silent sentence repairs without return must not become permanent second data stock.

PRACTICE IMPROVING AND CONTINUING

**From BUG-58 manual 1.1**

**To date**

Version 1.0: Check characters and period before handover.

**Newly released**

Version 1.1 · H06: After reimport check characters AND period display; check index annotation and dependent output from the same version.

The book production now needs a reliable handover. Samir therefore proposes an immediately implementable additional control: After the return import, the affected characters and period awards are checked. Nora sets out the technical criteria. Eva releases the transitional rule and explains it in the editorial round; in the manual it is now as H06.

At the same time, Eva commissions the revision of the comment tool. Samir and the development investigate export and reimport, receive the original annotation and complete a regression test. This takes more time than the additional examination. Until the new path is taken, the editorial team continues to work with the transitional rule.

Later, Lukas plays through his recipient correction in the revised tool: The untouched time is retained. Nora and Samir take the improved back-and-back path technically and technically. The additional control has secured the interim; the tool change removes the cause. Both were initiated and implemented.

The next manual change is also due. The team checks which additional control is still necessary after the tried and tested tool change. A transitional rule does not last forever, because it once made sense.

1. 1Findings and proposed amendments
2. 2Technical and technical review
3. 3approval and new test
4. 4Notification and application
5. 5Next joint examination

View the common manual ↑

---

▼ Toolbox for Samir's investigation

No tool has to do anything: Overview, XML examination and professional classification are interlocking.

Investigate a location

**Git pale, show and diff**

The line history provides an introduction to the last assigned change. Previous levels and differences can then be read specifically and connected with annotation, check deduction and correction note.

**Note:** No automatic cause identification. Deleted or replaced lines and previous changes require further comparisons. The associated name is not proof of individual guilt.

git blame · git show · git diff

First overview

**Git Quick Stats**

Displays statistics about commits, files and activity. Samir checks first which areas and time periods deserve a closer look.

**Note:** No TEI semantics. Imports, bots, renamings and mass formatting can shape the image; statistics do not explain its causes.

Project and application examples

Targeted report

**Gitinspector**

Creates historical repository reports and can be limited to selected file types. For TEI files, Samir XML must explicitly include in the file filter.

**Note:** Standard reports are person-oriented, XML is not included in the default file filter. A XML output file of the report is not an analysis of the TEI meaning.

Project and documentation

See on the team

**Gource**

Visualizes the development of a repository as a moving file tree. For a workshop, the growth and change of editing can be illustrated.

**Note:** A vivid animation is not a proof of quality. Names and activity can become visible; also the joint demonstration needs an clarified scope.

Project, examples and setup

Connect Git and TEI

**PyDriller**

A Python library to run commits and modified files. Samir can connect previous and later file contents with its own XML checks.

**Note:** Custom scripts and technical definitions are required. Built-in code/method metrics are not TEI quality measurement. Consider missing content, renamings and merge strategy.

Documentation on modified files · Project and start-up

Check the XML page

**Jing, Schematron, XPath and XSLT**

Jing checks against a RELAX-NG-schema. Schematron formulates additional rules. XPath selects elements and attributes; XSLT can generate structured comparisons, reports, and outputs.

**Note:** These are different tools and languages. Regular version, context and test run are documented. A formally valid reference can show the wrong technical position.

♪ Jing ♪ · Schematron · XPath

Complementary exploration of patterns

**CodeScene / Code Maat**

Hotspots or jointly modified files provide search directions. Samir compares them with the professionally classified correction cases.

**Note:** Code health assessments are not transferred to TEI. A frequently changed area can show intensive scientific work instead of poor quality.

CodeScene: Product and Tour · Code Maat

Where there is a greater need

**GrimoireLab**

Can merge data from Git, processes and review systems and access them as dashboards. Interesting when a pilot creates a well-founded permanent need.

**Note:** More setup and care; the linked data also allow deeper personal insights. Do not precautionably collect all available sources.

Project, supported sources and entry

---

▼ Which measures would be useful for the edition?

DevOps measures provide suggestions, not finished performance standards for edition work. These proposals require additional technical classification and test data.

**Late errors after one work step**

Confirmed errors, which are only noticeable after a transfer or typesetting proof, in relation to the comparable tested scope. Constant error classes and test density are important.

**Time from notification to verified remediation**

Start: documented error message. End: professionally checked correction. Waiting times, difficulty and responsibility; neither individual working hours nor automatic breach of deadlines.

**Reopened corrections**

Which messages have to be reopened? Group related follow-up errors so that the same chain does not appear as many independent problems.

**Amendment without a new reference test**

After a relevant text/comment change: Is there any evidence that the target and current edition have been checked together? Test proofs are additional data, not truth readable from Git.

**Jointly amended areas**

Which files or areas are frequently changed together? This can show a dependency or just a common formatting. A conspicuous coupling is a reason to look, not an error proof.

**Changes to TEI structures**

Which elements, attributes and relationships are actually changed? Consider mechanical formatting and mass changes separately. A structural comparison does not automatically recognize a change of meaning.

**Persistence of formal problems**

How long does a recognized schema or reference problem persist? Document test times and the respective valid standard versions; do not project a later tightened rule unnoticed back to earlier work as then error.

**Encoding of comparable cases**

Where do comparable points differ from the documented conventions? Consider material type and project phase. Distinguish technically justified exceptions from unintended inconsistencies.

DORA explains parameters of the software delivery. A correction time in the edition process is not automatically a DORA measure.

**Close Reading: understand the process**

Read text, characters, annotation and earlier versions of a passage in detail. git blame points to the last assigned line change; git show and comparisons open up further stands. Technical assignment alone explains neither cause nor responsibility.

**Distant Reading: Identify the pattern**

Compare many cases by process path, time and type of change. Accumulations, stable counter-examples and recurring correction loops show where to look more closely.

**Together: meaningful monitoring**

Switch from the pattern back to the location and from the declared point to other cases. Develop targeted tests and follow improvements. The possible depth of individual insights requires transparent rules.

---

▼ Five typical vulnerabilities in the workflow

**Error in the transmission route**

An export or return import changes relevant characters or structures. Check whether the agreed content survives the path unchanged; not just look at the source file.

**Problematic sequence**

A thorough examination is before a later changing step. Your statement applies to the old version; afterwards a targeted new control is necessary.

**Transfer with loss**

Assumptions and formats change between tools, teams or processing steps. Each handover is a possible checkpoint, not automatically a proven error.

**Error cascade**

An early change affects number recognition, index and multiple outputs. The visible correction at one point may require further revisions.

**Triggered antipattern**

A problematic procedure becomes a habit: repair only in the flag, do not reverse changes, repeatedly admonish for care without examining the transmission route.

NEW FRAMEWORK · DATA PROTECTION

#### When the analysis draws attention to people

So far, it has been a matter of error analysis, monitoring and improving the processes. Now another question arises: What personal insights arise – and who can use them for what?

**"Nevertheless, we should talk about our working hours**

The technical cause, the quick protection and the revision of the tool are discussed. Samir comes back to his entry: "Nevertheless: Should we perhaps talk about our working hours? Before the deadline it was quite late. And we could regularly track how long corrections remain.

"I'd rather work at night and do something else in the afternoon. I don't want to be measured by target times for comments. After all, we're not in the software industry."

Eva nods: "When you work best is your business. If we need such nights before each release, we should talk about dates and support. That's my responsibility."

"All right. Our agreement describes troubleshooting. Is this how regular evaluation is part of it? " Samir shakes his head. "No. And I shouldn't have made a concentration story out of the nocturnal entries. Before we make a routine out of it, we have to clarify questions and limits together. " His timestamps are not proof of working hours.

**Joint responsibility needs clear standards**

The employee agreement was explained at the start. It regulates contributions, accesses and the protection of the work history. Samir's new evaluation is not allowed at all. The team delimits the technical order: Which transfer changes the data? A regular observation of daily times does not belong to it.

It is also recommended to develop the standards for progress and quality together. In software development, such a common understanding is common in Scrum, for example: The team checks its workflow and agrees on improvements. For the edition, reliable handovers, expertly tested processing levels and less repeated post-corrections could count. Which measurement values fit this can be derived from the tasks of the edition; individual target times for scientific comments do not follow from this.

A long-lasting project will have to change its processes further. Together clarified purposes, accesses and limits facilitate these adjustments. Thus, Samir's precise insights can enable both technical improvements as well as a factual discussion about planning and relief. Legal and ethical limits remain binding.

Scrum Guide: joint improvement in the retrospective

**Responsible for:** Editorial, staff and, where appropriate, their representations, coordination, technical and legally responsible bodies

**Institutional question:** What rules, resources and common standards do the project need to ensure that process management and discussion about load can take place fairly?